GAMEPLAY DOMAIN & TACTICAL COMBAT

DOC ID: WC-ARCH-2026-VOL3

WildCard 게임 아키텍처 분석 보고서 (Vol. 3)

카드, 성유물, 전술스킬 도메인 및 헥사곤 전장 전투 아키텍처

프로젝트:	WildCard (Unreal Engine 5.7 Dedicated Server)	**작성 일시:**	2026-10-19
핵심 도메인:	PlayerCardComponent, ArtifactBase, TacticalSkillBase, EffectBase, TileMap, UnitBase	**수학/알고리즘:**	Hex Axial (Q, R) Coordinates, FRandomStream Seeded RNG, Billboard Lerp
문서 범위:	5대 카드 구역 관리, 성유물 내구도, 전술스킬 전략 패턴, UEffectBase 효과 파이프라인, 육각 타일 및 3D 빌보드 유닛		

1. 카드 시스템 및 5대 구역(Zone) 관리 아키텍처

1.1. UPlayerCardComponent와 카드 구역 수명주기

WildCard의 카드 수명주기는 AInGamePlayerState에 부착된 UPlayerCardComponent가 단독 관할합니다. 서버는 모든 카드를 인스턴스화된 UCardBase 객체로 추적하며, 아래의 **5대 카드 구역(Zone)** 간 이동으로 관리합니다.

구역 (ECardZone)	서버 컨테이너	클라이언트 가시성	구역의 역할 및 전이 조건
DECK	ServerDeck (TArray)	비공개 (DeckCount만 복제)	게임 시작 시 셔플된 미사용 카드 더미. 턴 시작 시 Hand로 1장 드로우.
HAND	ServerHand (TArray)	CurrentHandDisplay (소유자 한정)	플레이어가 손에 쥐고 있는 카드. 최대 핸드 수 제한 적용.
BATTLEFIELD	ServerBattlefield (TArray)	공개 (소환된 유닛 액터로 전장 표시)	유닛 카드가 타일 위에 실체화(Spawn)되어 배치된 상태.
EXILE (묘지)	ServerExile (TArray)	공개 (무덤 카드 장수)	주문 사용 후 소모되거나 전투 중 사망한 유닛 카드가 안식하는 구역.
OBLITERATE (추방)	ServerObliterate (TArray)	비공개/공개 (특수 효과)	부활이나 재사용이 불가능하도록 게임에서 영구 제외된 카드 구역.

1.2. 시드 기반 결정론적 난수(FRandomStream) 보장

네트워크 대전 게임에서 발생할 수 있는 비결정론적 버그 및 클라이언트-서버 간 난수 불일치를 방지하기 위해, WildCard는 C++ 표준 rand()나 언리얼의 글로벌 FMath::Rand()를 절대 사용하지 않습니다. 매치 생성 시 서버가 동기화된 시드로 FRandomStream을 초기화하여 덱 셔플 및 무작위 효과를 결정론적으로 제어합니다.

Source/WildCard/InGame/Game/PlayerCardComponent.cpp

C++ / UE 5.7

```cpp
void UPlayerCardComponent::ShuffleDeck()
{
    // 규칙: 반드시 서버 통제 하의 시드 난수 스트림(FRandomStream)을 사용하여 셔플
    int32 LastIndex = ServerDeck.Num() - 1;
    for (int32 i = 0; i <= LastIndex; ++i)
    {
        int32 Index = DeckRandomStream.RandRange(i, LastIndex);
        if (i != Index)
        {
            ServerDeck.Swap(i, Index);
        }
    }
    DeckCount = ServerDeck.Num();
}

UCardBase* UPlayerCardComponent::DrawCard(FName CardIDToDraw, int32 BaseMana)
{
    if (GetOwnerRole() != ROLE_Authority) return nullptr;
    if (ServerDeck.Num() == 0)
    {
        UE_LOG(LogWildCard, Warning, TEXT("[Card] 덱이 고갈되어 카드를 드로우할 수 없습니다."));
        return nullptr;
    }

    // 덱 최상단 카드 추출 후 손패 구역으로 이동
    UCardBase* DrawnCard = ServerDeck.Pop();
    ServerHand.Add(DrawnCard);

    HandCount = ServerHand.Num();
    DeckCount = ServerDeck.Num();

    // 소유자 클라이언트에 손패 표시 데이터 동기화
    SyncHandDataToDisplay();
    return DrawnCard;
}
```

2. 성유물 (Artifact) 시스템 아키텍처

성유물은 매치 준비 단계(Gameset)에서 플레이어가 덱과 직업에 맞춰 비밀리에 선택하는 영구 장비입니다. 기존 카드와 달리 필드에 유닛으로 배치되지 않고 플레이어 상태(InGamePlayerState)에 장착되며, 고유의 **내구도(Durability)**를 기반으로 작동합니다.

■ 성유물 라이프사이클 4단계

1. 비공개 선택 (Gameset)
플레이어가 3종의 성유물 중 1종 선택 (Server_SubmitArtifact). 서버 메모리에만 저장.

2. 상호 공개 (Mulligan)
SyncArtifactToDisplay를 통해 FWildArtifactDisplayData가 양측 클라이언트에 복제됨.

3. 트리거 발동 (UseArtifact)
턴 시작, 주문 시전, 유닛 피격 등의 트리거 시점에 UEffectBase를 호출하고 내구도 차감.

4. 비활성화 (Deactivate)
내구도가 0에 도달하면 bIsActive = false로 전환되어 효과 소멸.

Source/WildCard/InGame/Artifact/ArtifactBase.cpp — C++ / UE 5.7

```cpp
void UArtifactBase::UseArtifact(const FWildActionData& ActionData)
{
    if (!bIsActive || CurrentDurability <= 0) return;

    // 성유물에 등록된 모든 UEffectBase 효과 인스턴스를 순차 실행
    FEffectData DefaultEffectData;
    for (UEffectBase* Effect : ArtifactEffects)
    {
        if (Effect)
        {
            Effect->ExecuteEffect(ActionData, DefaultEffectData);
        }
    }

    // 내구도 1 소모
    CurrentDurability--;
    UE_LOG(LogWildCard, Log, TEXT("[Artifact] 성유물 %s 발동! 잔여 내구도: %d/%d"),
        *ArtifactID.ToString(), CurrentDurability, MaxDurability);

    if (CurrentDurability <= 0)
    {
        DeactivateArtifact();
    }
}

void UArtifactBase::DeactivateArtifact()
{
    bIsActive = false;
    UE_LOG(LogWildCard, Log, TEXT("[Artifact] 성유물 %s 내구도 고갈로 비활성화"), *ArtifactID.ToString());
}
```

3. 전술 스킬 (Tactical Skill) 전략 패턴 (Strategy Pattern)

WildCard는 전사, 마법사, 도사 각 직업별 전술 스킬의 고유한 시전 조건과 타겟팅 메커니즘을 **전략 패턴(Strategy Pattern)**을 통해 깔끔하게 추상화하였습니다.

직업 파생 클래스	스킬명	타겟팅 방식 (TargetType)	메커니즘 및 시스템 특징
UWarriorTacticalSkill	강타 (Heavy Strike)	TargetTile (인접 적 유닛)	인접 1칸 거리의 적에게 즉시 강력한 물리 피해를 입히고 타일 넉백 연출 트리거.
UMageTacticalSkill	비전 탄막 (Arcane Barrage)	TargetTile (사거리 3칸)	원거리 원소 마법 피해 및 연쇄 반응 효과.
MageSearchLibraryTacticalSkill	서고 탐색 (Search Library)	Self (패 모달 선택)	덱 최상단 카드 3장 중 1장을 선택하여 손패로 가져오는 비동기 모달 UI 연동 스킬.
UTaoistTacticalSkill	회복의 진 (Healing Formation)	TargetAllyUnit (아군 유닛)	자신 또는 아군 유닛의 체력을 회복시키고 정화 효과 부여.

3.1. 전술 스킬 다형성 인터페이스 (UTacticalSkillBase)

Source/WildCard/InGame/Unit/TacticalSkillBase.h C++ / UE 5.7

```cpp
UCLASS(Blueprintable, Abstract)
class WILDCARD_API UTacticalSkillBase : public UObject
{
    GENERATED_BODY()

public:
    UPROPERTY(EditDefaultsOnly, BlueprintReadOnly, Category = "Tactical Skill")
    FName SkillID;

    UPROPERTY(EditDefaultsOnly, BlueprintReadOnly, Category = "Tactical Skill")
    EWildPlayerClass RequiredClass;

    UPROPERTY(EditDefaultsOnly, BlueprintReadOnly, Category = "Tactical Skill")
    int32 ManaCost;

    UPROPERTY(EditDefaultsOnly, BlueprintReadOnly, Category = "Tactical Skill")
    int32 Range;

    UPROPERTY(EditDefaultsOnly, BlueprintReadOnly, Category = "Tactical Skill")
    ETacticalSkillTargetType TargetType;

    UPROPERTY(EditDefaultsOnly, BlueprintReadOnly, Category = "Tactical Skill|Visual")
    FGameplayTag VisualCueTag;

    // 클라이언트: 스킬별 모달 또는 3D 타겟팅 시작 진입점
    virtual void InitiateSkill(AInGameController* PC, APlayerUnit* SourceUnit);

    // 서버: 행동 선언 전 시전 가능 여부 검증
    virtual bool CanActivate(APlayerUnit* SourceUnit, ATile* TargetTile) const;

    // 서버: 실제 스킬 로직 실행 및 수치 적용
    virtual bool Execute(APlayerUnit* SourceUnit, ATile* TargetTile);

    // 체인 해결 시 클라이언트 플레이어의 비동기 추가 선택이 필요한지 여부 (OCP 다형성 위임)
    virtual bool RequiresPlayerPromptOnResolve(const FWildActionData& ActionData) const { return false; }

    // 체인 해결 시 클라이언트 플레이어에게 비동기 선택 프롬프트 요청
    virtual void PromptPlayerOnResolve(class AInGameController* PC, const FWildActionData& ActionData) {}
};
```

4. 헥사곤 전장 및 유닛(Unit) 아키텍처

4.0. 타일 상태 머신 및 강타입 ETileState 시스템

WildCard의 타일은 원시 정수(매직 넘버) 대신 UENUM(BlueprintType) enum class ETileState : uint8로 엄격하게 타입 안정성이 보장됩니다. C++ 레이어에서는 SetTileStateEnum(ETileState State) 헬퍼를 통해 타일 상태를 변경하며, 블루프린트(BP_Tile)의 SetTileState(int State) 이벤트와 100% 하위 호환됩니다.

상태 (ETileState)	정수값	전장 색상	설명 및 활성화 시점
Idle	0	기본 (어두운 회색)	일반 대기 타일. 활성화된 인터랙션 없음.
Combatable	1	빨간색 (Red Glow)	적 유닛이 위치하여 공격/전투 상호작용 가능한 타일.
Moveable	2	청록색 (Cyan)	선택된 아군 유닛이 이동 가능한 빈 타일.
Spawnable	3	주황색 (Orange)	비공개 메인 행동 또는 턴 시작 시 유닛을 소환 가능한 타일.
Effectable	4	보라색 (Purple)	주문 또는 특수 효과의 대상이 될 수 있는 타일.

4.1. 육각형 축 좌표계 (Axial Coordinates: Q, R)

WildCard의 전장은 정육각형 타일들의 집합으로 구성되며, 수학적으로 가장 효율적인 **축 좌표계(Axial Coordinates: (Q, R))**를 채택하고 있습니다. 세 번째 가상 입방 좌표 S는 항등식 $Q + R + S = 0$에 의해 $S = -Q - R$로 즉시 유도됩니다.

■ 헥사곤 좌표계 변환 및 최단 거리 수학 공식

① 입방 및 축 좌표계 항등식 (Cube ↔ Axial)

$$Q + R + S = 0 \quad \Longrightarrow \quad S = -Q - R$$

메모리에는 축 좌표 (Q, R)만 저장(FAxial)하고, 거리 계산 및 6방향 순회 시 입방체 (Q, R, S)로 무손실 즉시 변환(FCube).

② 헥사곤 맨해튼 최단 거리 (Hexagonal Distance)

$$D(A, B) = \frac{|Q_A - Q_B| + |R_A - R_B| + |S_A - S_B|}{2} = \max\left(|\Delta Q|, |\Delta R|, |\Delta S|\right)$$

두 타일 A, B 간 최단 이동 칸 수. ATileMap::GetTilesAtDistance의 링 탐색 및 사거리 판정의 수학적 기초.

③ Flat-Topped 월드 좌표 변환 (ATileMap::AddTile)

$$W = 2 \cdot \text{Size}, \quad H = \sqrt{3} \cdot \text{Size}$$
$$\text{Loc}_X = \text{Col} \cdot (0.75 \cdot W) = 1.5 \cdot \text{Col} \cdot \text{Size}$$
$$\text{Loc}_Y = \text{Row} \cdot H + \begin{cases} 0.5 \cdot H & (\text{Col} \,\%\, 2 \neq 0) \\ 0 & (\text{Col} \,\%\, 2 = 0) \end{cases}$$

3D 언리얼 공간상 평평한 상단(Flat-Top) 타일의 중심 액터 스폰 위치 $(X, Y, 0)$ 결정.

④ 6방향 인접 타일 오프셋 벡터 (Directions[6])

동 (E):	(+1, 0)	서 (W):	(-1, 0)
북동 (NE):	(+1, -1)	남서 (SW):	(-1, +1)
북서 (NW):	(0, -1)	남동 (SE):	(0, +1)

FCube::Directions 6개 단위 벡터: $(\pm 1, \mp 1, 0)$ 순환을 통해 모든 방향 일관된 이웃 탐색.

4.2. AUnitBase: 3D 카드 빌보드 및 클라이언트 부드러운 보간 이동

전장에 배치된 유닛은 3D 메시 대신 **카드 형태의 빌보드(Billboard)** 메시를 사용하여 TCG 특유의 시각적 감성을 전달합니다. EUnitBillboardMode를 통해 탑다운 카메라의 피치/요 회전에 맞춰 항상 카메라를 정면으로 응시(Camera-

Facing)하도록 회전 변환됩니다.

또한 서버에서 유닛 이동 패킷(OnRep_Axial 또는 이동 브로드캐스트)을 수신했을 때, 유닛이 순간이동(Teleport)하지 않고 부드럽게 타일 중심을 향해 이동하도록 **클라이언트 틱 보간 엔진(Client-side Smooth Interpolation)**이 내장되어 있습니다.

Source/WildCard/InGame/Unit/UnitBase.cpp C++ / UE 5.7

```cpp
void AUnitBase::StartVisualMove(const FVector& FromWorld, const FVector& ToWorld)
{
    VisualMoveStartLocation = FromWorld;
    VisualMoveTargetLocation = ToWorld;
    VisualMoveElapsed = 0.0f;
    VisualMoveDuration = 0.35f; // 0.35초 동안 부드럽게 이동
    bIsVisualMoving = true;
    SetActorLocation(FromWorld);
}

void AUnitBase::UpdateVisualMove(float DeltaTime)
{
    if (!bIsVisualMoving) return;

    VisualMoveElapsed += DeltaTime;
    float Alpha = FMath::Clamp(VisualMoveElapsed / VisualMoveDuration, 0.0f, 1.0f);

    // 부드러운 가감속을 위한 EaseInOut 커브 적용
    float SmoothAlpha = FMath::InterpEaseInOut(0.0f, 1.0f, Alpha, 2.0f);
    FVector NewLocation = FMath::Lerp(VisualMoveStartLocation, VisualMoveTargetLocation, SmoothAlpha);
    SetActorLocation(NewLocation);

    if (Alpha >= 1.0f)
    {
        bIsVisualMoving = false;
        SetActorLocation(VisualMoveTargetLocation);
    }
}
```

4.3. 마우스 상호작용 충돌 간섭 차단 (DisableMouseClickCollision)

3D 유닛 액터가 타일 위에 서 있을 때, 유닛 메시나 머리 위 위젯 컴포넌트가 플레이어의 마우스 레이캐스트를 차단하여 타일 클릭이 씹히는 문제(Click Deadlock)가 발생할 수 있습니다. 이를 원천 방지하기 위해 AUnitBase는 생성 시 모든 충돌 채널에서 마우스 트레이스를 투과(Ignore)시키는 전용 함수를 실행합니다.

Source/WildCard/InGame/Unit/UnitBase.cpp

C++ / UE 5.7

```cpp
void AUnitBase::DisableActorMouseClickCollision(AActor* Actor)
{
    if (!Actor) return;

    // 액터의 모든 프리미티브 및 위젯 컴포넌트 순회
    TArray<UPrimitiveComponent*> PrimitiveComponents;
    Actor->GetComponents<UPrimitiveComponent>(PrimitiveComponents);
    for (UPrimitiveComponent* Comp : PrimitiveComponents)
    {
        if (Comp)
        {
            // 카메라 및 가시성(Visibility) 트레이스 채널을 무시하여 타일 레이캐스팅 투과 보장
            Comp->SetCollisionResponseToChannel(ECC_Visibility, ECR_Ignore);
            Comp->SetCollisionResponseToChannel(ECC_Camera, ECR_Ignore);
        }
    }
}
```

5. 결론 및 요약

Volume 3에서는 5대 카드 구역과 결정론적 셔플, 성유물의 내구도 기반 생명주기, 직업별 전술스킬의 전략 패턴 구현, 헥사곤 축 좌표계 수학 및 3D 빌보드 유닛의 부드러운 보간 시스템을 규명하였습니다.

> **차기 볼륨 안내:**
> 마지막 **Volume 4**에서는 ModelViewViewModel(MVVM) 기반 반응형 UI 계층, Zero-Touch HUD 및 비주얼 큐(Visual Cue) 단일 이벤트 버스, 계층형 전투 로그, 그리고 Slate 기반 커스텀 헥사곤 에디터 툴링과 파이프라인 자동화 체계를 분석합니다.